

May 19, 2022 5:00

todo_complete.py

Page 1/2

```

1  import json
2  import os
3  import sys
4
5  from PySide6.QtCore import QAbstractListModel, Qt
6  from PySide6.QtGui import QImage
7  from PySide6.QtWidgets import QApplication, QMainWindow
8
9  from MainWindow import Ui_MainWindow
10
11 basedir = os.path.dirname(__file__)
12
13 tick = QImage(os.path.join(basedir, "tick.png"))
14
15
16 class TodoModel(QAbstractListModel):
17     def __init__(self, todos=None):
18         super().__init__()
19         self.todos = todos or []
20
21     def data(self, index, role):
22         if role == Qt.DisplayRole:
23             status, text = self.todos[index.row()]
24             return text
25
26         if role == Qt.DecorationRole:
27             status, text = self.todos[index.row()]
28             if status:
29                 return tick
30
31     def rowCount(self, index):
32         return len(self.todos)
33
34
35 class MainWindow(QMainWindow, Ui_MainWindow):
36     def __init__(self):
37         super().__init__()
38         self.setupUi(self)
39         self.model = TodoModel()
40         self.load()
41         self.todoView.setModel(self.model)
42         self.addButton.pressed.connect(self.add)
43         self.deleteButton.pressed.connect(self.delete)
44         self.completeButton.pressed.connect(self.complete)
45
46     def add(self):
47         """
48         Add an item to our todo list, getting the text from the QLineEdit .todoEdit
49         and then clearing it.
50         """
51         text = self.todoEdit.text()
52         # Remove whitespace from the ends of the string.
53         text = text.strip()
54         if text: # Don't add empty strings.
55             # Access the list via the model.
56             self.model.todos.append((False, text))
57             # Trigger refresh.

```

May 19, 2022 5:00

todo_complete.py

Page 2/2

```

58         self.model.layoutChanged.emit()
59         # Empty the input
60         self.todoEdit.setText("")
61         self.save()
62
63     def delete(self):
64         indexes = self.todoView.selectedIndexes()
65         if indexes:
66             # Indexes is a single-item list in single-select mode.
67             index = indexes[0]
68             # Remove the item and refresh.
69             del self.model.todos[index.row()]
70             self.model.layoutChanged.emit()
71             # Clear the selection (as it is no longer valid).
72             self.todoView.clearSelection()
73             self.save()
74
75     def complete(self):
76         indexes = self.todoView.selectedIndexes()
77         if indexes:
78             index = indexes[0]
79             row = index.row()
80             status, text = self.model.todos[row]
81             self.model.todos[row] = (True, text)
82             # .dataChanged takes top-left and bottom right, which are equal
83             # for a single selection.
84             self.model.dataChanged.emit(index, index)
85             # Clear the selection (as it is no longer valid).
86             self.todoView.clearSelection()
87             self.save()
88
89     def load(self):
90         try:
91             with open("data.json", "r") as f:
92                 self.model.todos = json.load(f)
93         except Exception:
94             pass
95
96     def save(self):
97         with open("data.json", "w") as f:
98             data = json.dump(self.model.todos, f)
99
100
101 app = QApplication(sys.argv)
102 window = MainWindow()
103 window.show()
104 app.exec()

```